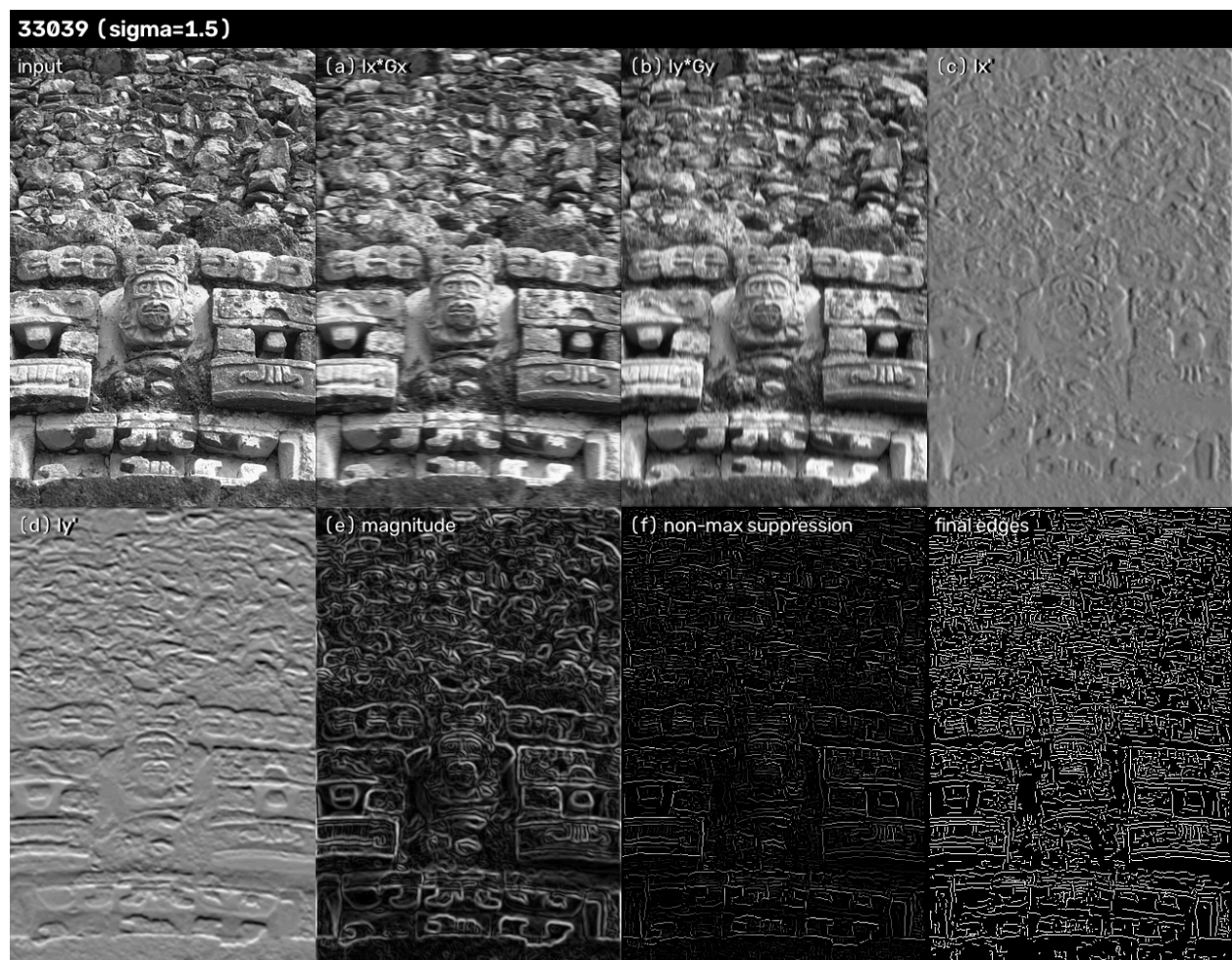
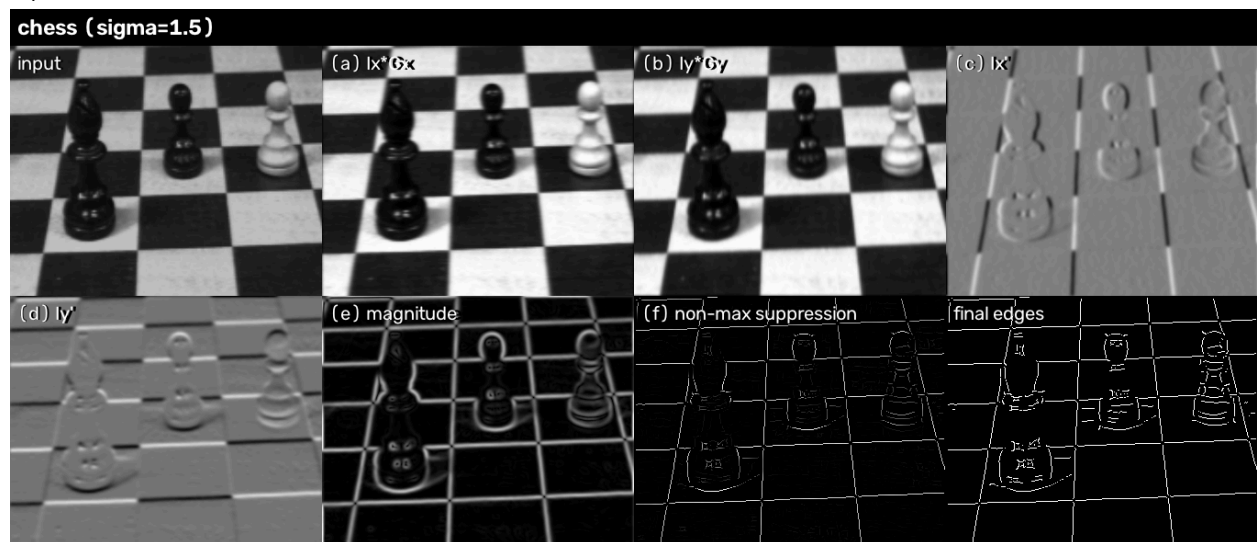


“A short write-up about your implementation with results: 1) A figure showing intermediate results as in Figure above, and 2) Similar Figures showing the effect of σ ”

Implementation Write Up:

My implementation of canny edge detection follows very closely to the outlined steps described in the assignment pdf. The first step I took was to write a function which reads in image paths and converts them to np arrays using opencv. Next was implementing a function to generate a 1d gaussian kernel which will be convolved with the images to smooth them before detecting edges. This smoothing improves the resulting edge detection as extreme changes between pixels are reduced and edges are made clearer as a result. We first used the provided sigma value to determine the size of our gaussian kernel. This size is three times the size of sigma. This rule is called the 65-95-99 rule and is based on the concept that 99.7% of all data points are 3 standard deviations away from the center. So larger sigma values will result in a larger kernel and more intense smoothing occurring. Once this kernel is generated we want to find the first derivative which can be found simply by multiplying the gaussian kernel by $-x/\sigma^2$. Next we need to convolute these kernels with our images so I created a function to ingest an img and a kernel and perform convolution with the 1d kernel depending on the axis passed into the function call (“horizontal” or “vertical”). With these functions defined we can now perform convolution with our images and our gaussian kernels in either the X direction or the Y direction to produce the specified images which can be seen below. After this we compute the magnitude which is a simple 1 line operation thanks to the power of numpy arrays. After that we implement NMS which will ensure we remove any unnecessary edges and shrink the thickness of edges to be more true to the real edge. This is done by computing the angle of the gradient vector at each pixel then identifying the edges' neighbors based on its gradient angle. Then we keep the pixel if it is larger than both its neighbors and remove otherwise. Lastly we apply hysteresis thresholding to decide which of the remaining pixels are really edges. This uses two thresholds that are calculated from the largest magnitude value in the image. Any pixel above the high threshold is marked as a strong edge and any pixel between the low and high thresholds is marked as a weak edge, with the low threshold being just a fraction of the high threshold. A weak pixel on its own is probably just noise so we only keep it if it is connected to a strong pixel somewhere along the same edge. To figure this out I used OpenCV's `connectedComponents` function which allows me to determine if the weak pixels are actually connected to stronger pixels or not. All these generated images are then displayed and saved using opencv's `write` and `show` functionality.

1)



2)

